

Data & Domain Integration — Brief for Tiyasha

Project: ORCA (SIH26176) — Marine EcOsystem Reasoning with Collaborative Agents

Your track: Data & Domain Integration

Closest collaborators: Samprikta (builds the agents that call your functions) · Dhrubojyoti (architecture, will review your interfaces)

This document is yours to keep in your project folder and hand to any AI assistant you work with — it's written to give both you and that assistant full context in one read.

Chapter 1 — Why This Track Exists

ORCA's entire pitch rests on one claim: the answers it gives a fisherman are real, not generated-sounding guesses. Every agent in this system is, underneath, a language model reasoning over whatever numbers it's handed. If those numbers are fake, stale, or wrong, the whole product is fake — no amount of good prompting fixes bad data.

You're the one making that claim true. Everyone downstream of you — the agents, the demo, the pitch — trusts what you hand them without re-checking it.

Chapter 2 — What You're Actually Building

Three things. Resist the urge to expand this list:

1. A working connection to **MOSDAC** (ISRO's ocean data portal) — sea surface temperature, chlorophyll, ocean current.
2. A working connection to **INCOIS** — Potential Fishing Zone advisories and hazard/weather alerts.
3. A **geofencing dataset** — maritime boundaries, marine protected areas, restricted zones.

Each becomes a small set of clean Python functions — not raw API calls scattered through the codebase, but a fixed "menu" that Samprikta's agents can call without knowing or caring how MOSDAC or INCOIS work underneath.

Chapter 3 — Research First, Before Any Code

- Read MOSDAC's Data Download API documentation and INCOIS's ERDDAP documentation end to end. Not skimmed — you need to know what parameters each expects, what shape the response comes back in, whether an account or key is required, and what happens when you ask for something that doesn't exist (a bad date, a point with no coverage).
- Pull one real response from each source manually and actually look at it before writing any function. If you don't understand the raw response yourself, no function you build around it will be trustworthy.
- Check whether either source needs registration or an API key — sort that out now, not as a Week 2 surprise.
- Check for rate limits or usage terms worth knowing before the team depends on this live, in front of judges.
- Write down anything confusing, inconsistent, or undocumented as you go. You'll need this list — see Chapter 6.

Chapter 4 — Before You Start a Project in Antigravity

Put together a small reference folder first, and have it ready to hand to whatever AI assistant you work with inside Antigravity:

- The official ORCA problem statement (SIH26176)
- `TECH_STACK.md` — the team's locked technical decisions (get it from Dhrubojyoti)
- Whatever documentation, PDFs, or notes you gathered in Chapter 3
- The raw sample responses you pulled manually — save them as actual files, not just something you glanced at once
- This brief

An AI assistant grounded in real reference material will help you far more reliably than one working from a verbal description alone. Feed it these files up front.

Chapter 5 — Folder Structure You're Building Toward

```
data/
  tools/
    mosdac_tools.py
    incois_tools.py
    geofence_tools.py
  raw_samples/          <- your manual pulls from Chapter 3
  demo_fallback_cache.json <- see Chapter 7
docs/
  data-sources.md        <- your running log, Chapters 3 & 6
  eval-queries.md
tests/
  test_data_tools.py
```

Each tool function should take plain, obvious inputs (a location, a date, a sector name) and return one predictable structure — something in the shape of `get_sst(lat, lon, date)` or `get_pfz_zones(sector)`. The exact structure is your call. Once you've decided it, treat it as a contract: don't quietly change field names on Samprikta partway through.

Chapter 6 — How to Work, Not What to Type

- Get one function working and returning sane output for one input before worrying about edge cases. Manual first, automated second.
- Log every quirk you hit — a date format that didn't match the docs, a field that's sometimes missing, units that surprised you. `docs/data-sources.md` will matter more than your code the day something breaks mid-demo.
- Standardize units early. Ocean data sources are inconsistent about Celsius vs. Kelvin, UTC vs. IST, coordinate formats. Decide the team's standard now and convert at the boundary, not all over the codebase.
- Agree your function's return shape with Samprikta before you consider it final, not after she's already built against it.
- Whatever an AI assistant scaffolds for you, read and understand it before accepting it. You're the one who'll be asked "why does this return that" in Week 3, not the assistant.
- Leave yourself room to try things — a couple of ways of structuring the geofence data, for instance, before locking one in. That's a decision worth testing, not guessing at.

Chapter 7 — Things Worth Knowing That Might Not Have Come Up Yet

- **Start your fallback cache in Week 1, not Week 5.** The moment you get one good real pull from each source, save it as a static file. That's your safety net for the live demo, and it lets the rest of the team build against realistic data before your live functions are fully ready.
- **Pick 2-3 coastal regions to focus on — not all of India's coastline.** A demo that works flawlessly for a couple of real regions beats one that half-works everywhere. Confirm the choice with Dhrubojyoti early, since it decides how much geofence data you actually need to gather.
- **Never commit real credentials.** If MOSDAC needs login details, keep that file out of git entirely and commit an example/template instead.
- **Re-read the brief's eight sample queries every so often.** It's easy to build data functions that are technically correct but don't actually answer what the problem statement is asking. Check your work against those queries, not just against whether the API call succeeded.
- **Flag any ISRO/INCOIS data-usage or attribution terms to Adhiraj** — he'll need them for the pitch and the idea submission write-up.

Chapter 8 — Context for an AI Assistant

If you're an AI assistant reading this for context: Tiyyasha owns the data layer for ORCA, a multi-agent marine information system built for SIH 2026. Her three deliverables are typed, tested Python functions wrapping MOSDAC (ISRO ocean data), INCOIS (fishing-zone and hazard advisories), and a geofencing dataset. These functions are consumed by separate Weather/Ocean/Risk agents built by a teammate — so stable signatures and clear typing matter more than cleverness.

`TECH_STACK.md` holds the team's full technical decisions and takes precedence over any assumption you'd otherwise default to. Help her build, test, and document — but which data sources to trust, how to structure the schema, and what the raw API responses actually mean should stay her judgment calls, not something decided for her.